

JavaScript Notes — Day: Increment/Decrement, Precedence, Strings & Prompts

Source: A Smarter Way to Learn JavaScript, Ch. 6–9 + app.js practice

1. Increment & Decrement Operators

The Basics

num++ is shorthand for **num = num + 1** — it increases the variable by 1. **num--** is shorthand for **num = num - 1** — it decreases the variable by 1.

```
var num = 5;
num++;
console.log(num); // 6
```

Prefix vs. Postfix — the part that trips people up

Where you put the ++ or -- (before or after the variable) changes what value gets used *in that statement*, even though the variable itself always ends up incremented/decremented by 1.

Form	What happens	Example
num++ (postfix)	Returns the ORIGINAL value first, then increments num afterward	var num=1; var n2=num++; → n2 is 1, num is 2
++num (prefix)	Increments num FIRST, then returns the new value	var num=1; var n2=++num; → both n2 and num are 2
num-- (postfix)	Returns the ORIGINAL value first, then decrements num afterward	var num=1; var n2=num--; → n2 is 1, num is 0
--num (prefix)	Decrement num FIRST, then returns the new value	var num=1; var n2=--num; → both n2 and num are 0

Rule of thumb: **postfix (num++)** = "use it, then change it". **Prefix (++num)** = "change it, then use it".

Trace-through from app.js practice

```
var a = 1;
console.log(a++); // logs 1 (original value returned)
console.log(++a); // logs 3 (a was 2, prefix bumps to 3 first)

var a = 1;
console.log(--a); // logs 0 (prefix: decrement first)
console.log(a--); // logs 0 (postfix: return original, then a becomes -1)
console.log(a); // logs -1
```

```
var num = 1;
var num2 = num++; // num2 = 1, num = 2
console.log(num2); // logs 1
```

Common mistake to avoid: assuming `num++` and `++num` always behave the same because "num ends up +1 either way." That's true for the variable, but NOT true for the value returned into the expression/`console.log` — that's the part that decides what gets printed or assigned.

2. Math Expressions & Operator Precedence

Just like in algebra, multiplication and division are completed before addition and subtraction. When an expression is ambiguous, JavaScript's precedence rules decide the order — but the safer, more readable habit is to use parentheses to make the order explicit rather than relying on memorized precedence rules.

```
var totalCost = 1 + 3 * 4; // 13 (3*4 happens first)
var totalCost = 1 + (3 * 4); // 13 (parens make intent explicit)
var totalCost = (1 + 3) * 4; // 16 (parens force addition first)

var result = (2 * 4) * 4 + 2; // 34
var result = ((2 * 4) * 4) + 2; // 34 (explicit version of above)
var result = (2 * 4) * (4 + 2); // 48 (different grouping!)
```

Takeaway: parentheses always win over any built-in precedence rule, and they make code easier for you (and anyone reading it later) to understand at a glance — worth the extra keystrokes.

3. Concatenating Text Strings

The `+` operator joins (concatenates) strings together when at least one operand is a string. You can combine string literals, variables, or a mix of both.

```
alert("Thanks, " + userName + "!");

var message = "Thanks, ";
var userName = "Susan";
var banger = "!";
var customMess = message + userName + banger;
alert(customMess); // "Thanks, Susan!"
```

Numbers vs. strings — the part that causes real bugs

```
alert("2" + "2"); // "22" (both are strings → concatenation)
alert("2 plus 2 equals " + 2 + 2); // "2 plus 2 equals 22"
// JS converts the numbers to strings
// and concatenates left to right
```

Note the space inside `"Thanks, "` — without it you'd get `"Thanks,Susan!"` with no space after the comma. Small formatting details like this matter in concatenation.

4. Prompts

A prompt box asks the user for input and returns their response as a string — which you must capture in a variable.

```
var spec = prompt("Your species?", "human");  
// 2nd argument = optional default text shown in the field
```

- Unlike alert, a prompt needs a variable to capture the response.
- The second argument (optional) is the default answer pre-filled in the field.
- **Every** response comes back as a STRING, even if the user typed a number.
- Empty input + OK → variable is assigned an empty string "".
- Clicking Cancel → variable is assigned the special value null.

The classic string-vs-number prompt bug

```
var numberOfCats = prompt("How many cats?");  
var tooManyCats = numberOfCats + 1;  
// If user enters 3, you might expect tooManyCats = 4.  
// Actual result: "31" – because "3" is a string, so + concatenates  
// instead of adding. Fix: convert with Number() first.
```

This is exactly why app.js practice code wraps prompt results in Number():

```
var age = prompt("Enter your age: ");  
age = Number(age); // convert string → number  
console.log(age + 2); // now does real addition, not concatenation
```

Style note from the book: prefer the short form **prompt(...)** over **window.prompt(...)** (both work, short form is more common). Use double quotes for strings as the default convention.

5. Today's Practice File — app.js Walkthrough

Quick trace of the trickier lines from today's practice file, applying the rules above:

Code	Output / Result	Why
a=2; b=4; a=1; b=0; console.log(a*2+b);	2	a and b get reassigned before use — final values a=1, b=0 → 1*2+0
var a=1; a+=3; console.log(a);	4	+= adds and reassigns in one step
var a=7; a++; console.log(a);	8	++ as its own statement just increments
var a=1; console.log(a++); console.log(++a);	1, then 3	postfix logs old value (1), a becomes 2; prefix increments to 3 then logs it
var a=1; a++; ++a; console.log(a++); console.log(a);	3, then 4	a: 1→2→3, then postfix logs 3 and bumps to 4, final log is 4

The last two blocks in app.js (prompt for name, prompt for age) are direct applications of Section 4 — the age prompt correctly uses Number(age) to avoid the string-concatenation bug.

Quick Recap

- Postfix (`x++`) returns the old value, then updates. Prefix (`++x`) updates first, then returns.
- Use parentheses to make arithmetic order explicit — don't rely on memorized precedence.
- `+` concatenates when either side is a string; numbers get converted to strings automatically.
- `prompt()` always returns a string — use `Number()` to do real math on the result.

Notes generated from today's app.js practice + A Smarter Way to Learn JavaScript, Ch. 6–9.